NAME:PRENA .M REG NO:192524056
COURSE NAME:JAVA PROGRAMMING COURSE CODE:CSA 0924

CO4-AT1

A factory-floor control panel needs a perfectly uniform grid of 12 identically-sized indicator buttons, 3 rows by 4 columns, that all resize together proportionally when the operator maximizes the window. Which layout manager guarantees uniform cell sizing here, and why would FlowLayout fail to preserve the grid structure on resize?

AIM:

To design a factory-floor control panel with 12 identically-sized indicator buttons arranged in a 3 x 4 grid using GridLayout, which guarantees uniform cell sizing on resize, and to explain why FlowLayout fails to preserve the grid structure when the window is maximized.

PSEUDOCODE:

```
START
CREATE a JFrame named ControlPanel
SET its layout manager to GridLayout(3, 4)
    // 3 rows, 4 columns, equal-sized cells
FOR i = 1 to 12
    CREATE a JButton labeled "Indicator i"
    ADD the button to the frame
END FOR
SET the frame size and make it visible
WHEN the window is resized or maximized
    GridLayout recalculates all cells to equal
    width and height, keeping the 3x4 grid uniform
STOP
```

JAVA CODE:

```
import javax.swing.*;
import java.awt.*;

class ControlPanelDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Factory Control Panel");

        // GridLayout(rows, columns, hgap, vgap)
        // guarantees all 12 cells stay equal in size
        frame.setLayout(new GridLayout(3, 4, 5, 5));

        for (int i = 1; i <= 12; i++) {
            JButton indicator = new JButton("Indicator " + i);
            frame.add(indicator);
        }

        frame.setSize(500, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

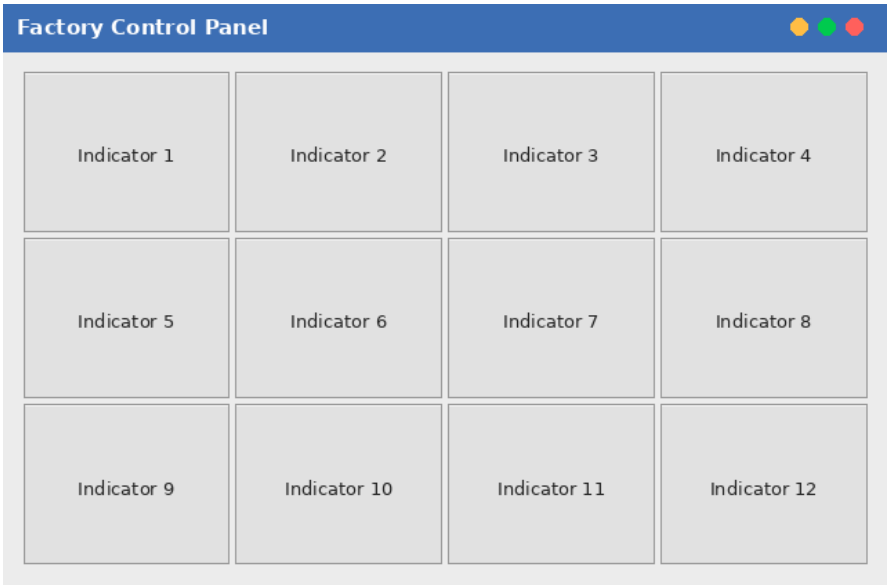
INPUT:

No user input required.

OUTPUT:

A window opens showing 12 buttons labeled Indicator 1 to Indicator 12, arranged in 3 rows and 4 columns. All buttons are exactly the same width and height. When the window is resized or maximized, every button grows or shrinks together so the 3x4 grid structure and equal cell sizes are preserved.

(This is a Swing GUI program, so it opens a window rather than printing console text; below is a mockup of the resulting window layout.)



RESULT / EXPLANATION:

GridLayout divides the container into a fixed number of rows and columns of exactly equal size, so all 12 indicator buttons are forced to remain identical in width and height no matter how the window is resized; each cell simply scales proportionally. FlowLayout, on the other hand, does not divide the container into a grid at all - it simply lays components out left to right in their natural preferred size and wraps to a new line only when a row runs out of horizontal space. This means the number of buttons per row changes unpredictably as the window is resized, the buttons do not stretch to fill the available space, and the neat 3x4 grid structure breaks apart into a variable, non-uniform arrangement. Hence GridLayout is the correct choice for a uniform, proportionally resizable 3x4 indicator panel, while FlowLayout cannot guarantee that structure.